

Failure Handling, Reliability, and Metrics

1. Failure Scenarios & Resilience Strategies

Distributed event architectures must withstand broker node crashes, transient network drops, and partner gateway downtime:

Failure Scenario	System Impact	Mitigation Strategy
Kafka Broker Failure	Broker node crashes or disk fails	Topics are replicated across 3 brokers with <code>min.insync.replicas = 2</code> . If a leader crashes, an in-sync replica is elected with zero message loss.
Network Partition	Intermittent drop between producer and Kafka	Producers buffer events locally in memory and retry with backoff. Consumers auto-reconnect and resume from their last committed offset.
Duplicate Events	Re-delivered message after consumer timeout	Consumers check Redis for the unique <code>eventId</code> before processing. If already seen, the message offset is committed and execution skipped.
Out-of-Order Events	Cancellation arriving before confirmation	All events use <code>bookingId</code> as the Kafka partition key, guaranteeing that related state transitions land on the same partition and process in order.

2. Offset Tracking, Idempotency & Replay Mechanisms

Consumer Offset Tracking

- Offsets are committed to Kafka only **after** the notification has been successfully delivered by the channel worker or safely handed off to the Dead-Letter Queue (`notifications-dlq`).
- This guarantees at-least-once delivery with zero dropped customer alerts.

Idempotency Keys

- Every event carries a globally unique `eventId` generated at origin.
- Consumers write `eventId` to Redis with a 24-hour TTL (`EX 86400`) upon successful dispatch, preventing duplicate SMS, push, or email sends.

Event Replay Mechanism

- Kafka retains event logs for 7 days. If a downstream telecom gateway suffers an extended outage, operations engineers can rewind the consumer group offset to a specific timestamp (`--to-datetime`) to cleanly replay and re-deliver missed notifications once connectivity is restored.

3. Observability Stack & Centralized Logging

- Centralized Log Aggregation (ELK):** All microservices, Kafka brokers, and notification workers emit structured JSON logs ingested by Logstash into Elasticsearch.
- End-to-End Correlation IDs:** Every incoming customer transaction carries a unified `eventId` and `userId` propagated across all asynchronous Kafka headers. Engineers can query Kibana with a single `eventId` to trace a message from `BookingService` through Kafka topics to final Twilio or SES delivery.

4. Key Performance Metrics

Metric Category	Target Component	Measurement & Purpose	Healthy Threshold
Event Throughput	Kafka Consumers	Total events consumed and routed per second across all topics.	> 10,000 events/sec
Delivery Success Rate	Channel Workers	Percentage of successful deliveries for Email, SMS, Push, and WebSockets.	> 99.5% success
Consumer Queue Lag	Kafka Partitions	Number of unconsumed messages waiting in topic partitions. Rising lag triggers worker pod autoscaling.	Lag < 500 messages
Retry & DLQ Ingestion	Error Queues	Rate of messages undergoing exponential retries versus routed permanently to notifications-dlq.	DLQ rate < 0.1%

5. Service Level Objectives (SLOs) & Alert Rules

Primary Booking Confirmation Latency SLO

- **Objective:** 99% of booking confirmation notifications (Push, SMS, and Email) must be delivered to users within **5.0 seconds** of payment completion.
- **Alert Rule:** If p99 delivery latency exceeds 5.0 seconds for more than 2 consecutive minutes, Prometheus Alertmanager triggers a **P1 Critical Alert** to the on-call engineer.

Secondary Show Reminder Timeliness SLO

- **Objective:** 95% of scheduled show reminders must be dispatched within **1.0 minute** of their target trigger timestamp.
- **Alert Rule:** If reminder consumer lag exceeds 500 messages or dispatch delay exceeds 60 seconds, an alert fires to scale up reminder worker pods.

6. Incident Escalation & Auto-Remediation Pipeline

Pipeline: Event Failure & Degradation Response

1. Metric Threshold Breach: Prometheus detects rising lag or DLQ surge (> 500 messages).

2. Automated Pod Scaling: Horizontal Pod Autoscaler (HPA) spins up additional channel worker pods.

3. Provider Outage Fallback: If external gateway error rate > 20%, traffic auto-switches to secondary provider (e.g., Twilio -> AWS SNS).

4. Engineer Paging: Alertmanager dispatches P1 alert to PagerDuty and incident response channel.

5. Replay & Reconciliation: Once external partner recovers, failed DLQ messages are replayed cleanly.